



Universidad Nacional de La Matanza

Departamento de Ingeniería e Investigaciones Tecnológicas

Ingeniería en informática

02-2900: Paradigmas de Programación

Trabajo Práctico N°2: Recital

Profesores

- Aubin, Veronica Inés
- Gasior, Federico Mauro
- Lanzilotta, Hernán Gabriel
- Videla Lucas

Grupo Omicron

Integrantes

- Iván Enrique Coarite Coarite
- Diego Martin Estevez
- Matías José Daniel Guliano
- Joel Alexis Rueda Olarte
- Lautaro Vega

Fecha de entrega: Martes, 25 de Noviembre de 2025

Índice

Índice	2
Introducción.....	3
Desarrollo.....	4
Requisitos funcionales	4
Diagrama de clases	6
Conclusiones.....	7
Referencias	8
Librerías utilizadas	8
Documentación utilizada para el proyecto	8
Especificaciones del formato de los datos	8
Enlaces a archivos del proyecto.....	8

Introducción

El presente proyecto aborda el diseño e implementación de un sistema de gestión para la organización de un recital, integrando múltiples tecnologías con el paradigma orientado a objetos y el lógico. El objetivo principal es modelar el dominio del problema (artistas, canciones, roles, requisitos y configuración del recital) y desarrollar una aplicación capaz de importar datos para el recital, validar información, asignar roles de forma consistente y permitir consultas mediante una base de conocimiento lógica.

Para ello empleamos un enfoque híbrido: por un lado, se utiliza Java como lenguaje orientado a objetos para representar el modelo de dominio, implementar la lógica de negocio, gestionar las operaciones de importación desde archivos JSON y brindar una interfaz de interacción a través de un menú. Por otro lado, recurrimos a Prolog como lenguaje lógico con el fin de expresar relaciones, restricciones y reglas propias del conocimiento del recital, permitiendo verificar condiciones y realizar consultas mediante su motor de resolución.

La integración entre ambos lenguajes se realiza mediante JPL, permitiendo que la aplicación Java consulte dinámicamente una base de conocimiento generada a partir de los datos cargados. Además, el diseño del sistema se apoyó en un diagrama de clases UML, que permitió estructurar y visualizar de forma clara las relaciones, responsabilidades y dependencias entre las entidades del dominio. El diagrama se encuentra acompañado de una breve descripción de cada clase que lo compone, junto con sus responsabilidades e interrelaciones que permitieron confeccionarlo.

El sistema resultante combina conceptos de programación imperativa, orientada a objetos y lógica, ofreciendo una solución para la gestión de eventos musicales. Además, el desarrollo del proyecto permitió aplicar principios de modularidad, abstracción, pruebas unitarias e integración con archivos externos, consolidando los contenidos trabajados durante la materia.

Desarrollo

El desarrollo del sistema se basó en los requisitos funcionales y técnicos establecidos en la consigna del trabajo práctico, orientado a modelar y resolver la problemática de organizar un recital mediante los conceptos del paradigma orientado a objetos.

A partir del análisis del dominio se diseñó un modelo representado mediante un diagrama de clases y posteriormente se implementó el sistema, integrado con Prolog para consultas lógicas específicas.

Requisitos funcionales

El sistema debía cumplir con una serie de funcionalidades que permiten administrar el proceso de preparación del recital. Estas funcionalidades coinciden con las operaciones disponibles en el menú principal de la aplicación, tales como:

- **Detección de roles faltantes**

- Identificar qué roles faltan para una canción específica.
- Identificar los roles faltantes para todas las canciones del recital.

Estas operaciones deben considerar los artistas base disponibles, reduciendo la cantidad de roles a cubrir por contratación.

- **Contratación de artistas**

- Contratar artistas externos para una canción específica, minimizando el costo total.
- Contratar artistas para todas las canciones del recital simultáneamente, aplicando descuentos por bandas compartidas y respetando la cantidad máxima de canciones que cada artista puede interpretar.

Los roles ya contratados en pasos previos deben respetarse y no ser sobrescritos.

- **Entrenamiento de artistas**

- El sistema debe permitir entrenar artistas para adquirir nuevos roles, incrementando su costo.
- No se pueden entrenar artistas base ni artistas ya contratados.

- **Gestión del estado del recital**

- Listar artistas contratados y sus datos relevantes.
- Listar canciones y su estado (roles cubiertos, faltantes, costo).

- Quitar artistas de canciones, de todas las canciones o del lineup general.
- Guardar el estado actual del recital en un archivo.
- Cargar un estado previo desde archivo.
- **Integración con Prolog:** Se debía incorporar una consulta lógica capaz de responder: ¿Cuántos entrenamientos mínimos se requieren para cubrir todos los roles del recital utilizando únicamente artistas base y artistas sin experiencia con costo uniforme?

Para ello se genera y consulta dinámicamente una base de conocimiento mediante JPL.

Respecto a los requisitos deseables (Aquellos que no imprescindibles para el correcto funcionamiento del sistema, pero que cuya implementación resulta en una mejora del mismo), se implementó la **persistencia** del recital (El ya mencionado guardado y carga de recitales), y el **arrepentimiento** (Quitar artistas ya contratados).

[illegible]

Aunque la finalidad de este documento no es ser un informe detallado del diagrama de clases, es importante mencionar las responsabilidades y funciones de las clases fundamentales del sistema:

Menu: Clase encargada de proporcionar la interfaz gráfica para que el usuario final pueda interactuar con el sistema.

IntegranteDeUnRol: Controla de que en una canción los artistas asignados a un rol no sean mayor de los que se necesita.

La clase **IntegracionProlog** actúa como el módulo encargado de vincular el sistema Java con Prolog. Su responsabilidad principal es generar automáticamente la base de conocimiento Prolog a partir de los archivos JSON del proyecto (artistas, recital y discográfica), convirtiendo esa información en hechos y reglas que luego serán utilizados por el motor lógico. Además, proporciona un método para consultar Prolog desde Java, permitiendo obtener el cálculo de los entrenamientos mínimos requeridos para cubrir todos los roles del recital. En conjunto, esta clase encapsula toda la lógica relacionada con la comunicación y sincronización entre ambos lenguajes, manteniendo aislada esta

funcionalidad del resto del sistema. Es notable mencionar que se podría realizar una más profunda separación de incumbencias entre una clase que genere la base de conocimiento y otra que determine las reglas y predicados que hacen a la consulta, pero a los fines del trabajo, agrupamos ambas funciones en una sola clase.

Conclusiones

Este trabajo nos permitió integrar Java y Prolog en un mismo proyecto, algo que al principio resultó complejo pero que terminó resultando en una experiencia muy útil. Logramos construir un sistema capaz de gestionar recitales, artistas y entradas, y que se apoya en una base de conocimiento generada automáticamente desde Java y consultada con Prolog. Esta combinación nos permitió entender mejor cómo se comunican ambos lenguajes y cómo organizar un proyecto para que esa interacción funcione correctamente.

En el proceso surgieron varios problemas técnicos (desde la configuración de Maven hasta errores de JPL y la carga de recursos, como los archivos JSON), pero resolverlos nos ayudó a afianzar conceptos y a mejorar la arquitectura del sistema. También pudimos implementar los puntos bonus, como la funcionalidad de arrepentimiento, que le da al sistema una capa extra de comodidad y control para el usuario.

En general, el proyecto nos permitió aplicar contenidos de la materia en un caso práctico completo, trabajando con archivos, lógica, pruebas, modularización y buenas prácticas. El resultado final es un sistema funcional, ordenado y que cumple con los requisitos planteados, a la vez que resultó en un aprendizaje considerable para futuros desarrollos donde haya que combinar distintos paradigmas o tecnologías.

Referencias

Librerías utilizadas

Google. (2024). *Gson: A Java serialization/deserialization library to convert Java Objects into JSON and back* (Version 2.13.2). <https://github.com/google/gson>

JUnit Team. (2024). *JUnit 5 User Guide* (Version 5.11.3). <https://junit.org/junit5/>

Fitch, J., Wielemaker, J., & SICS. (2024). *JPL: Java Interface for SWI-Prolog* (Version 7.4.0). SWI-Prolog. <https://jpl7.org/>

Wielemaker, J. (2024). *SWI-Prolog Reference Manual*. SWI-Prolog Solutions b.v. <https://www.swi-prolog.org/>

Documentación utilizada para el proyecto

JPL Development Team. (2024). *Java calls Prolog – JPL Tutorial*. <https://jpl7.org/TutorialJavaCallsProlog>

SWI-Prolog. (2024). *JPL Standalone Examples – Family.java*. <https://github.com/SWI-Prolog/packages-jpl>

Paulo, A. (2021, septiembre 6). *Error org.jpl7 .JPLException unsupported blob type passed from Prolog*. Stack Overflow. <https://stackoverflow.com/questions/68596288/error-org-jpl7-jplexception-unsupported-blob-type-passed-from-prolog>

Dev, U. (2023, julio 5). *Maven Java lang ClassNotFoundException: Could not find or load main class*. Stack Overflow. <https://stackoverflow.com/questions/78175722/maven-java-lang-classnotfoundexception-could-not-find-or-load-main-class>

Especificaciones del formato de los datos

Bray, T. (2017). *The JavaScript Object Notation (JSON) Data Interchange Format* (RFC 8259). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8259>

Enlaces a archivos del proyecto

Grupo Omicron. (2025). *ParadigmasDeProgramacion-TrabajoPracticoDos*. GitHub. <https://github.com/joelruedaalexis/ParadigmasDeProgramacion-TrabajoPracticoDos.git>